



24 - PUZZLE

Tarea 2

Instituto Tecnológico de Culiacán

Inteligencia Artificial

José Mario Ríos Félix

18-19

02/03/26

Roberto Valenzuela Arce

22170843

Instrucciones

1. Descripción del Problema

El objetivo es implementar un agente inteligente capaz de resolver el 24-Puzzle (tablero de 5x5). El sistema debe encontrar una secuencia de movimientos que lleve un estado inicial aleatorio al estado objetivo (orden ascendente con el espacio vacío al final).

2. Algoritmo Sugerido: IDA* (Iterative Deepening A*)

Debido a la explosión combinatoria de un tablero de 5x5 ($5! \times 4! \times 3! \times 2! \times 1! = 120 \times 24 \times 6 \times 2 \times 1 = 34560$ permutaciones), se requiere el uso del algoritmo IDA*. El estudiante deberá justificar por qué este algoritmo es superior a A* en términos de gestión de memoria para este espacio de estados.

3. Requisitos de la Heurística

El programa debe permitir alternar entre, al menos, dos funciones heurísticas para comparar su eficiencia:

Distancia de Manhattan: La suma de las distancias horizontales y verticales de cada pieza a su posición final.

Conflicto Lineal (Opcional/Extra): Una extensión de la Distancia de Manhattan que añade penalizaciones cuando dos piezas están en su fila/columna correcta pero en el orden inverso.

4. Entregables Técnicos

Código Fuente: Implementación limpia en [Insertar Lenguaje, ej. Python/C++].

Análisis de Rendimiento: Una tabla que compare:

Número de nodos expandidos.

Tiempo de ejecución.

Longitud de la solución (número de movimientos).

Demostración: El programa debe aceptar una configuración inicial manual o generar una aleatoria que sea resoluble.

1. Justificación: ¿Por qué IDA* y no A* para el 24-Puzzle?

Si usamos el algoritmo A* que hicimos en la Tarea 1 en un tablero de 5x5, el programa se va a estrellar con un error de OutOfMemoryError en cuestión de minutos.

- **El problema de A (Memoria Exponencial):** A* guarda en memoria todos los nodos generados (en la PriorityQueue) y todos los nodos visitados (en el HashSet). En un 5x5, la frontera de búsqueda crece de forma exponencial. Tu memoria RAM se llenará antes de llegar a la solución.
- **La solución de IDA (Memoria Lineal):** Iterative Deepening A* (IDA*) combina la heurística inteligente de A* con la eficiencia de memoria de la Búsqueda por Profundidad (DFS). En lugar de guardar todos los nodos explorados, IDA* solo guarda el camino actual que está explorando. Su consumo de memoria es $O(d)$ (donde d es la profundidad máxima), lo que significa que requiere poquísima memoria RAM, sin importar qué tan grande sea el tablero.

Análisis de Rendimiento: 24-Puzzle (Estado a 41 movimientos)

Estado Inicial:

```
7, 1, 4, 10, 3,
0, 6, 9, 8, 5,
11, 2, 12, 14, 15,
17, 21, 13, 23, 20,
16, 22, 19, 18, 24
```

Estado Objetivo:

```
1, 2, 3, 4, 5,
6, 7, 8, 9, 10,
11, 12, 13, 14, 15,
16, 17, 18, 19, 20,
21, 22, 23, 24, 0
```

Métrica	Heurística 1: Distancia de Manhattan	Heurística 2: Manhattan + Conflicto Lineal	Mejora / Diferencia
Estimación Inicial (h)	27	31	+4 (Más informada)
Nodos Expandidos	1,772,899	280,304	- 84.1% nodos explorados
Tiempo de Ejecución	100 ms	103 ms	+ 3 ms (Costo por cálculo extra)
Longitud de la Solución	41 movimientos	41 movimientos	Ambas son Admisibles (Óptimas)

Tabla comparativa 8-Puzzle (Estado a 12 movimientos)

```
EstadoInicial = "1238 4765";  
EstadoObjetivo = "123478 65";
```

Métrica	1. Anchura (BFS)	2. Profundidad (DFS)	3. Prioridad (A* con Manhattan)
Nodos Extraídos (Analizados)	2,088	123,454	154 (Más eficiente)
Estados en Memoria (Visitados)	3,293	171,264	250 (Menor consumo RAM)
Longitud de la Solución (Pasos)	12 (Óptimo)	55,014 (Subóptimo)	12 (Óptimo)

Los resultados demuestran la superioridad de la heurística de Conflicto Lineal en la gestión de memoria y espacio de búsqueda del algoritmo IDA. Al añadir una penalización para las piezas invertidas linealmente, la estimación inicial se acercó más al costo real (31 vs 27), lo que permitió evadir iteraciones tempranas e inútiles en el árbol de búsqueda. Como resultado, el número de nodos expandidos se redujo drásticamente en un 84%. Aunque el tiempo de ejecución fue similar debido a la sobrecarga computacional (overhead) de calcular las colisiones en filas y columnas, la reducción en el consumo de memoria hace que Conflicto Lineal sea mucho más escalable para configuraciones iniciales de mayor profundidad.

Código:

```
import java.util.Arrays;  
  
public class Main {  
    public static void main(String[] args) {  
        byte[] objetivo = {  
            1, 2, 3, 4, 5,  
            6, 7, 8, 9, 10,  
            11, 12, 13, 14, 15,  
            16, 17, 18, 19, 20,  
            21, 22, 23, 24, 0  
        };  
  
        Puzzle24 puzzle = new Puzzle24(objetivo);  
  
        // Generamos un estado inicial aleatorio (mezclado 45 veces)  
        byte[] inicial = //puzzle.generarAleatorioResoluble(45);  
        {  
            7, 1, 4, 10, 3,
```

```

        0, 6, 9, 8, 5,
        11, 2, 12, 14, 15,
        17, 21, 13, 23, 20,
        16, 22, 19, 18, 24
    };

    System.out.println("Estado Inicial Generado:");
    puzzle.imprimirTablero(inicial);

    // -----
    // *** SELECCIONA LA HEURÍSTICA A EVALUAR ***
    // 1 = Distancia de Manhattan pura
    // 2 = Manhattan + Conflicto Lineal (Extra/Optimizada)
    // -----
    int tipoHeuristica = 2;

    if (tipoHeuristica == 1) {
        System.out.println("--- EJECUTANDO IDA* CON MANHATTAN ---");
    } else {
        System.out.println("--- EJECUTANDO IDA* CON MANHATTAN +
CONFLICTO LINEAL ---");
    }

    puzzle.resolverIDASStar(inicial, tipoHeuristica);
}
}

class Puzzle24 {
    private byte[] objetivo;
    private long nodosExpandidos;

    private static final int[] DIR_X = {-1, 1, 0, 0};
    private static final int[] DIR_Y = {0, 0, -1, 1};

    public Puzzle24(byte[] objetivo) {
        this.objetivo = objetivo;
    }

    public void resolverIDASStar(byte[] inicial, int tipoHeuristica) {
        nodosExpandidos = 0;
        long inicioTiempo = System.currentTimeMillis();

        // Calcular heurística inicial dependiendo de la opción elegida
        int limiteF = calcularHeuristica(inicial, tipoHeuristica);
        System.out.println("Límite estimado inicial (h): " + limiteF);
    }
}

```

```

        while (true) {
            int resultado = busquedaDFS(inicial, 0, limiteF,
encontrarVacio(inicial), -1, tipoHeuristica);

            if (resultado == 0) {
                long finTiempo = System.currentTimeMillis();
                System.out.println("\n¡SOLUCIÓN ENCONTRADA!");
                System.out.println("=====
==");
                System.out.println("Heurística usada:      " +
(tipoHeuristica == 1 ? "Manhattan" : "Manhattan + Conflicto Lineal"));
                System.out.println("Nodos expandidos:      " +
nodosExpandidos);
                System.out.println("Tiempo de ejecución:  " + (finTiempo -
inicioTiempo) + " ms");
                System.out.println("Movimientos óptimos:  " + limiteF);
                System.out.println("=====
==");
                return;
            }
            if (resultado == Integer.MAX_VALUE) {
                System.out.println("No hay solución.");
                return;
            }

            limiteF = resultado;
            // Descomenta la siguiente línea si quieres ver cómo va subiendo
el límite en tiempo real
            System.out.println("Aumentando límite de búsqueda a: " +
limiteF);
        }
    }

    private int busquedaDFS(byte[] estado, int g, int limiteF, int posVacio,
int movAnterior, int tipoHeuristica) {
        int h = calcularHeuristica(estado, tipoHeuristica);
        int f = g + h;

        if (f > limiteF) return f;
        if (h == 0) return 0;

        int minExceso = Integer.MAX_VALUE;
        int xVacio = posVacio % 5;
        int yVacio = posVacio / 5;

```

```

        for (int i = 0; i < 4; i++) {
            if (esMovimientoOpuesto(i, movAnterior)) continue;

            int nuevoX = xVacio + DIR_X[i];
            int nuevoY = yVacio + DIR_Y[i];

            if (nuevoX >= 0 && nuevoX < 5 && nuevoY >= 0 && nuevoY < 5) {
                nodosExpandidos++;
                int nuevaPosVacio = nuevoY * 5 + nuevoX;

                intercambiar(estado, posVacio, nuevaPosVacio);
                int resultado = busquedaDFS(estado, g + 1, limiteF,
nuevaPosVacio, i, tipoHeuristica);
                if (resultado == 0) return 0;
                if (resultado < minExceso) minExceso = resultado;
                intercambiar(estado, posVacio, nuevaPosVacio); //
Backtracking
            }
        }
        return minExceso;
    }

    // Método selector de heurística
    private int calcularHeuristica(byte[] estado, int tipo) {
        if (tipo == 1) {
            return heuristicaManhattan(estado);
        } else {
            return heuristicaConflictoLineal(estado);
        }
    }

    // --- HEURÍSTICA 1: MANHATTAN ---
    private int heuristicaManhattan(byte[] estado) {
        int distancia = 0;
        for (int i = 0; i < 25; i++) {
            int valor = estado[i];
            if (valor != 0) {
                int filaActual = i / 5;
                int colActual = i % 5;
                int filaObjetivo = (valor - 1) / 5;
                int colObjetivo = (valor - 1) % 5;
                distancia += Math.abs(filaActual - filaObjetivo) +
Math.abs(colActual - colObjetivo);
            }
        }
    }

```

```

    }
    return distancia;
}

// --- HEURÍSTICA 2: MANHATTAN + CONFLICTO LINEAL ---
private int heuristicaConflictoLineal(byte[] estado) {
    int h = heuristicaManhattan(estado);
    int conflictos = 0;

    // Revisar conflictos en Filas
    for (int fila = 0; fila < 5; fila++) {
        for (int c1 = 0; c1 < 4; c1++) {
            for (int c2 = c1 + 1; c2 < 5; c2++) {
                int val1 = estado[fila * 5 + c1];
                int val2 = estado[fila * 5 + c2];

                if (val1 != 0 && val2 != 0) {
                    int filaObj1 = (val1 - 1) / 5;
                    int filaObj2 = (val2 - 1) / 5;

                    // Si ambos pertenecen a esta misma fila
                    if (filaObj1 == fila && filaObj2 == fila) {
                        int colObj1 = (val1 - 1) % 5;
                        int colObj2 = (val2 - 1) % 5;
                        // Si la pieza que está a la izquierda (val1)
                        debería ir a la derecha de val2
                        if (colObj1 > colObj2) conflictos++;
                    }
                }
            }
        }
    }

    // Revisar conflictos en Columnas
    for (int col = 0; col < 5; col++) {
        for (int f1 = 0; f1 < 4; f1++) {
            for (int f2 = f1 + 1; f2 < 5; f2++) {
                int val1 = estado[f1 * 5 + col];
                int val2 = estado[f2 * 5 + col];

                if (val1 != 0 && val2 != 0) {
                    int colObj1 = (val1 - 1) % 5;
                    int colObj2 = (val2 - 1) % 5;

                    // Si ambos pertenecen a esta misma columna

```

```

        if (colObj1 == col && colObj2 == col) {
            int filaObj1 = (val1 - 1) / 5;
            int filaObj2 = (val2 - 1) / 5;
            // Si la pieza de arriba (val1) debería ir abajo
de val2

            if (filaObj1 > filaObj2) conflictos++;
        }
    }
}

// Se suman 2 movimientos por cada conflicto lineal encontrado
return h + (2 * conflictos);
}

// --- MÉTODOS AUXILIARES ---
private void intercambiar(byte[] estado, int i, int j) {
    byte temp = estado[i];
    estado[i] = estado[j];
    estado[j] = temp;
}

private int encontrarVacio(byte[] estado) {
    for (int i = 0; i < 25; i++) {
        if (estado[i] == 0) return i;
    }
    return -1;
}

private boolean esMovimientoOpuesto(int movActual, int movAnterior) {
    if (movAnterior == -1) return false;
    if (movActual == 0 && movAnterior == 1) return true;
    if (movActual == 1 && movAnterior == 0) return true;
    if (movActual == 2 && movAnterior == 3) return true;
    if (movActual == 3 && movAnterior == 2) return true;
    return false;
}

public byte[] generarAleatorioResoluble(int movimientosAleatorios) {
    byte[] clon = Arrays.copyOf(objetivo, objetivo.length);
    int posVacio = 24;
    int movAnterior = -1;

    for (int i = 0; i < movimientosAleatorios; i++) {
        int xVacio = posVacio % 5;

```

```

        int yVacio = posVacio / 5;
        int mov, nuevoX, nuevoY;
        do {
            mov = (int) (Math.random() * 4);
            nuevoX = xVacio + DIR_X[mov];
            nuevoY = yVacio + DIR_Y[mov];
        } while (nuevoX < 0 || nuevoX >= 5 || nuevoY < 0 || nuevoY >= 5
|| esMovimientoOpuesto(mov, movAnterior));

        int nuevaPos = nuevoY * 5 + nuevoX;
        intercambiar(clon, posVacio, nuevaPos);
        posVacio = nuevaPos;
        movAnterior = mov;
    }
    return clon;
}

public void imprimirTablero(byte[] estado) {
    for (int i = 0; i < 25; i++) {
        if (estado[i] == 0) System.out.print(" _ \t");
        else System.out.print(estado[i] + "\t");
        if ((i + 1) % 5 == 0) System.out.println();
    }
    System.out.println();
}
}

```